Helios: can an LLM's confidence be trusted with real money?

Luke Puzenski · 10th grade, Thomas Jefferson High School for Science and Technology (Class of 2029) · October 2026
All AI runs locally on two RTX 3090 GPUs (Qwen 2.5, with embeddings on CPU), with no cloud AI APIs. About 49,000 lines of Python, written almost entirely by Claude Code to my design, with 974 tests and 730 commits since June 2026.

Helios is a swing-trading system I built. It runs every day on language models hosted on my own machine. A 72B model writes each investment thesis and rates how confident it is from 1 to 10, and separate, tested code decides whether the trade is allowed and how big it can be. It has traded real money since August 2026. Every closed trade is scored against SPY, so over time the record will show whether the model's confidence means anything. I'd like advice on how to test that properly.

Research questions

- Does an LLM's stated conviction predict realized returns? Each prompt includes the model's own win rate and alpha at each conviction level, and tells it to calibrate to that record "not against how confident you feel." The answer may be no.
- How do you test an LLM trader that may have memorized the past? Qwen's pretraining almost certainly includes these stocks' prices and news, so Helios never backtests and is judged only on live decisions.

How a trade happens

1. Events come in from SEC filings, options flow, news, insider buys, analyst ratings, macro data, congressional trades and arXiv, polled every 5 minutes to daily.
2. A small local model, Qwen 2.5 7B, decides whether each event is relevant and urgent.
3. For events that pass, the system builds a brief with price action, fundamentals, catalysts, a market-regime estimate, and earlier theses on the same stock from a vector store.
4. Qwen 2.5 72B writes the thesis as strict JSON with an action (open, add, hold, trim, exit, watch or pass), a 1 to 10 conviction, four 1 to 5 sub-scores, and upside and downside estimates. A buy needs conviction 5 or higher and upside at least twice the downside, and the bar rises for small accounts, so the live account needs a 7.
5. Plain code then sizes the position, sets a volatility-based stop and enforces the limits below. It can shrink a trade, reject it or stop all new buying, and the model can't change it.
6. Orders go to Interactive Brokers, long-only US stocks, and stops are checked every 30 minutes.
7. Every thesis, risk decision and fill is logged, slippage included. Trades are reviewed against their thesis at 30, 60 and 90 days, and alpha against SPY (priced at entry) is recorded at exit.

Rules the model can't override

About 1,500 lines of unit-tested code. Once, a small model mislabeled a stock's risk bucket and loosened its own limits, so the bucket now comes from a watchlist I maintain by hand.

Drawdown	No new buying after a 10% drop from the high-water mark (sells and stops keep running)	Entry size	At most 12%, 7% or 2% of the account, by risk bucket (higher while the account is very small)
Risk	2% of equity per trade, 7% total open risk	Concentration	35% per sector, 60% AI infrastructure, 10% speculative, and at most 20% of remaining cash per trade
Correlation	Size cut by a quarter when a stock is correlated 0.7 or more with two or more holdings	Stops	ATR-based by bucket, plus trailing stops

Open problems (where advice would help most)

- There's no good offline test for an LLM whose training data may include the test period. Forward testing is honest but slow, and this is the problem I most want to work on.
- The live sample is small (six buys, four closed), so the calibration question has little statistical power so far. Costs beyond measured slippage aren't modeled.
- Keeping the broker and the database in sync has been the hardest engineering problem. Orders that went unreconciled have left positions without protective stops more than once. The fixes are recent.
- It's long-only, mostly in AI infrastructure and chips, and untested through a regime change.